

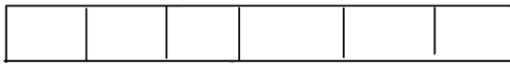
Why Hashmap are designed?

Before HashMap, JAVA Had Hashtable (Thread-Safe But Slow)

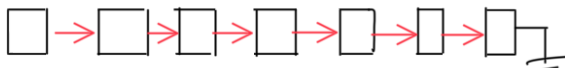
HashMap Was designed to

1. Provide Fast data retrieval using Hashing
2. Allow null keys (only 1) and (key is unique) Multiple null Values.
3. Be non-synchronized (faster in single-threaded apps)
4. Serve as a general-purpose implementation of Map Interface.

Lets take Array and Linked list as Example

Array 

→ Array Contains the students of a class

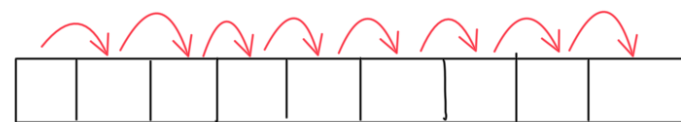
LinkedList 

→ linked list contains the students of a class

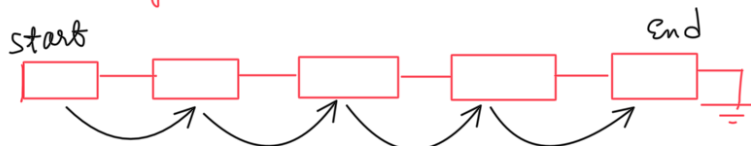
For Example
Name of
students of
class

Search Operation on these Data Structure

Search for a Student

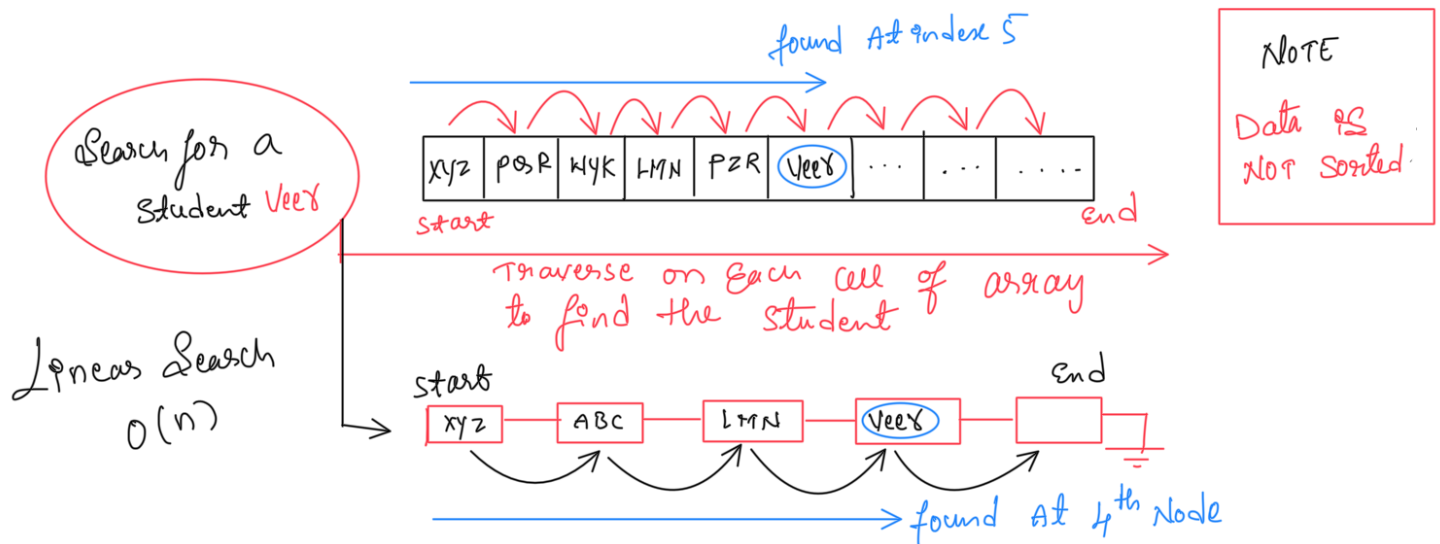


Traverse on Each cell of array to find the student



To Search an element the time complexity will be $O(n)$ where n is the length of array or the size of linked list.

We have to start from the 1st index of array / 1st Node of list to the last index of array / last Node of array for searching any element.



So in these above two data structures (Array, list) we have to perform search operation by linear search and the time complexity of search operation is $O(n)$

If the data is unsorted then we have to perform the linear search.

If in case the data is sorted then we can perform binary search which has time complexity $O(\log n)$

To optimize the searching operation the HashMap data structure is designed. (In Python it's called as Dictionary)

What is HASHMAP In JAVA ?

• `HashMap < K, V >` is a part of `java.util` package

- It is a Key-Value Pair data structure (like a dictionary in python, or map in C++)
- Each key is unique, But values can be duplicated
- It allows $O(1)$ average time complexity for insertion, deletion and lookup

Example

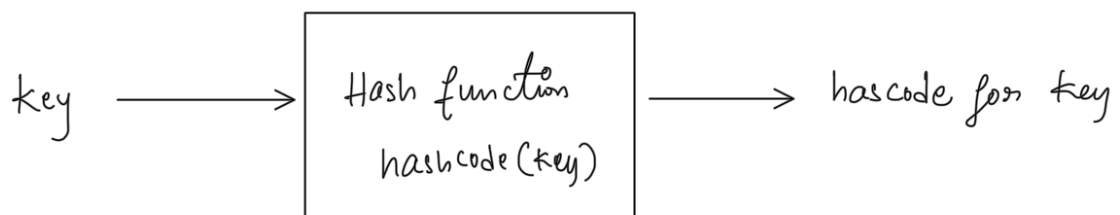
```
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 1);
map.put("B", 2);
map.put("C", 3);
```

```
System.out.println(map.get("B"));
```

How HashMap Works Internally

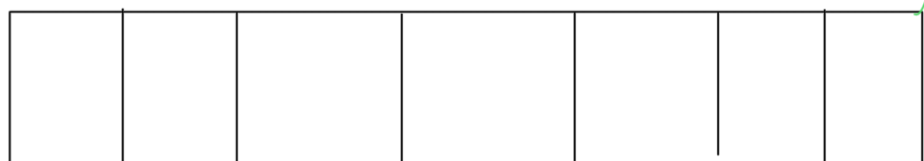
Step 1: Hashing

- When you insert a key-value pair (`put(key, value)`)
`put(K key, V value)`, the key is hashed using `hashCode()`.
- Then HashMap applies hash function to map the hash code into an array index (called a Bucket)



$$\text{Hashcode of key} \% \text{Capacity} = \text{index}$$

Array



Array of Buckets

$$\text{index} = \text{hashCode(key)} \% \text{Capacity}$$

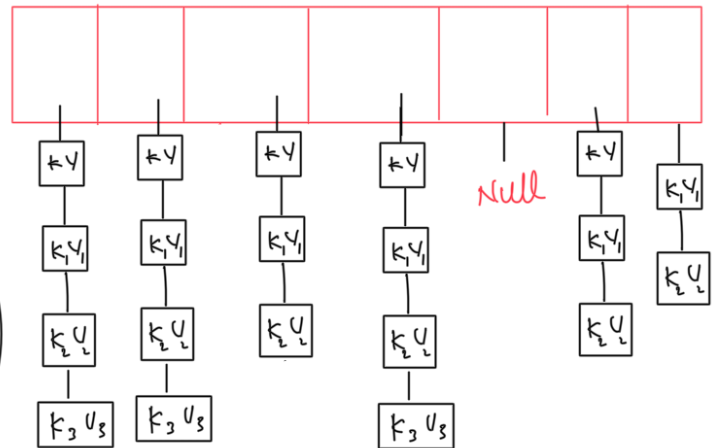
$$\text{Array size} = \text{Capacity}$$

step 2 : Buckets

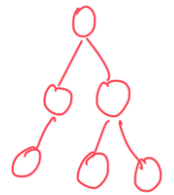
- Internally, HashMap maintains an array of Buckets
- Each Bucket can store multiple entries (key-value pairs)

To perform operation
with $\log n$ TC in
worst case

also
Red Black tree
can be used
in the Buckets



Array of
Buckets



uses tree in
case of more
collision

linked list of different key value pairs

step 3 : Handling collisions

When two keys Hash to the Same Bucket

1. Before Java 8 $\rightarrow$ HashMap used a Linked List inside the Bucket

- Worst case lookup $O(n)$

2. After Java 8 $\rightarrow$ if a Bucket grows beyond a threshold (usually 8), HashMap switches to a balanced Red-Black Tree

- Worst-case lookup $O(\log n)$

step 4 : Resizing

- Default capacity = 16 buckets

- Load factor = 0.75 (75% full)

- When entries exceeds capacity $\times$ loadFactor, the HashMap doubles its capacity and rehashes elements

Step 5 : Retrieving Values

get(key)

1. Compute Hash for key, find Bucket.
2. Traverse Bucket (Linked List or Tree)
3. Compare keys with Equals() until found
4. Return Value

Internal Structure (Java 8 onwards)

text

```
HashMap
|-- array of Node<K, V> (buckets)
    |-- Node: {int hash, K key, V value, Node next}
    |-- If too many nodes → Converted into a TreeNode (Red-Black Tree)
```

Time Complexity

Operation	Average Case	Worst Case
put(key, value)	$O(1)$	$O(\log n)$ (if tree) / $O(n)$ (if poor hash)
get(key)	$O(1)$	$O(\log n)$ / $O(n)$
remove(key)	$O(1)$	$O(\log n)$ / $O(n)$

HashMap Internal Structure

